Pixel vs. Symbolic Observations for PPO on Super Mario Bros (NES)

CSC-52081 — Reinforcement Learning Project Report

Lucas Martim | Sergio Contente | Leonardo Falabella | Gabriel Corsi | Lara Polachini | CSC-52081 | March 17, 2026

Abstract

*We compare two observation modalities under **Proximal Policy Optimization (PPO)** for learning to play Super Mario Bros: (1) raw pixel frames processed by a Nature CNN, and (2) a compact 13×16 symbolic grid extracted directly from NES RAM, fed to an MLP. A symbolic DQN baseline fails to converge (0% flag rate), motivating the switch to PPO. Our symbolic PPO agent achieves a **93% flag-reach rate on World 1-1** (peak 97%), running entirely on CPU at 500 fps—roughly 6× faster throughput than pixel PPO on GPU. Transfer learning (1-1 → 1-2) reaches 78% flag rate on the new level but triggers **catastrophic forgetting** of the source task. Random-start curriculum training reduces positional overfitting, while joint multi-task training across Worlds 1-1 to 1-4 stalls due to reward imbalance.*

1. Introduction

Super Mario Bros is a standard benchmark for deep reinforcement learning (RL): it demands long-horizon planning, precise motor control, and generalization across varied layouts. The canonical deep RL approach uses pixel observations processed by a convolutional network [2], but this pipeline is computationally expensive.

Our work asks two questions. **First**, does PPO [1] outperform vanilla DQN on this domain? **Second**, can a compact *symbolic* representation—tile IDs decoded from NES RAM—match pixel-based performance while dramatically reducing training cost?

We run six experiments: (1) a symbolic DQN baseline, (2) pixel PPO, (3) symbolic PPO on World 1-1, (4) transfer learning to World 1-2, (5) random-start curriculum training, and (6) multi-task training across Worlds 1-1 through 1-4.

2. Background

2.1. Deep Q-Networks

DQN [2] approximates $Q(s, a; \theta)$ with a deep network stabilized via a replay buffer and a target network:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\big)^2\Big]. \quad (1)$$

On long-horizon, semi-sparse tasks like Mario, ε-greedy exploration rarely discovers the flag, and Q-value overestimation from Eq. (1) causes divergence before meaningful credit assignment can occur.

2.2. Proximal Policy Optimization

PPO [1] is an on-policy actor-critic method that optimizes a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)\,\hat{A}_t\Big)\Big], \quad (2)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and $\hat{A}_t$ is the GAE advantage estimate [1]. Parallelism across N environments provides diverse, low-correlation rollouts without a large replay buffer, and GAE propagates sparse rewards effectively through long trajectories.

3. Methodology

3.1. Observation Representations

Pixel (CnnPolicy)

Frames are max-pooled (anti-flicker), converted to grayscale, resized to 84×84, and stacked over 4 timesteps. The $84 \times 84 \times 4$ tensor is processed by a Nature CNN [2] (3 conv. layers + FC 512).

Symbolic/RAM (MlpPolicy)

We read the NES RAM via `nes-py` and decode a 13×16 tile grid encoding five classes: empty, solid, enemy, Mario, power-up. Stacking 4 frames and appending the power-state byte yields an 833-dimensional float32 vector fed to a two-layer MLP $[512, 512]$.

3.2. Reward Shaping

Both pipelines share the same shaping scheme:

$$r = \frac{1}{10}\left(r_{\text{env}} + \frac{\Delta\text{score}}{40} + 50 \cdot \mathbf{1}[\text{flag}] - 50 \cdot \mathbf{1}[\text{death}]\right), \quad (3)$$

where r_{env} is the horizontal-progress signal. The 1/10 scale keeps advantages in a numerically stable range.

3.3. Training Setup

All PPO runs use 8 parallel environments (`SubprocVecEnv`) via Stable-Baselines3 [3] with a two-phase learning rate schedule. Table 1 lists key hyperparameters. The symbolic agent runs on CPU (8 cores, ≈500 fps); the pixel agent uses a Colab T4 GPU (≈78 fps).

Table 1: PPO hyperparameters (both agents).

Parameter	Value
γ (discount)	0.90
LR Phase 1 / Phase 2	2.5×10^{-4} / 10^{-5}
n_steps, batch size	512, 256
n_epochs	4
Clip ϵ, entropy	0.1, 0.01
Parallel environments	8

4. Experiments & Results

4.1. DQN Baseline

Symbolic DQN was trained for 200 k steps. It achieved 0% flag rate: ε-greedy exploration disrupts the sustained action sequences Mario requires, the replay buffer fills with correlated failures, and noisy Q-targets from Eq. (1) destabilize learning before any flag reward is encountered. This motivates the use of PPO for all subsequent experiments.

4.2. World 1-1: Pixel vs. Symbolic PPO

Symbolic PPO achieved **93% flag rate** (peak 97%) on World 1-1 with mean reward $\approx$304, converging in $\approx$500 k steps ($\sim$17 min on CPU). **Pixel PPO** reached $>$80% flag rate with mean reward $\approx$285, requiring substantially more wall-clock time due to CNN overhead and lower frame throughput on GPU.

Figure 1 shows the qualitative learning dynamics. The symbolic agent benefits from a 34× observation compression (833 vs. 28 224 floats), enabling $\approx$6× higher throughput and faster wall-clock convergence to a higher reward.

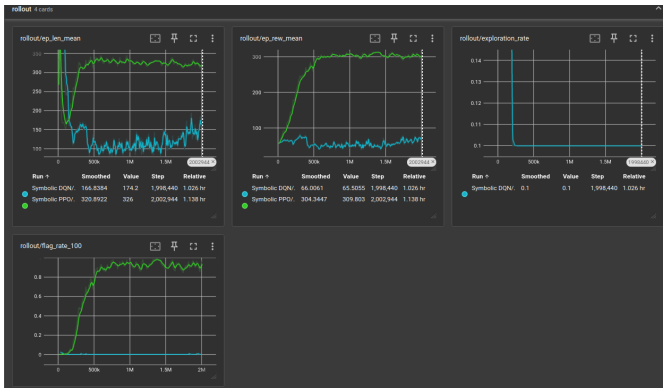

Figure 1: Symbolic PPO vs Symbolic DQN

4.3. Transfer Learning: 1-1 → 1-2

The trained Symbolic PPO model was fine-tuned on World 1-2 at reduced learning rate (10^{-5}, weights unfrozen).

- World 1-2 flag rate converged to **78%**, faster than training from scratch.
- World 1-1 flag rate collapsed to **0%**—a textbook case of *catastrophic forgetting* [4].

Because PPO is on-policy, World 1-1 transitions are discarded during fine-tuning, and gradient updates on 1-2 data overwrite the policy weights specialized for 1-1.

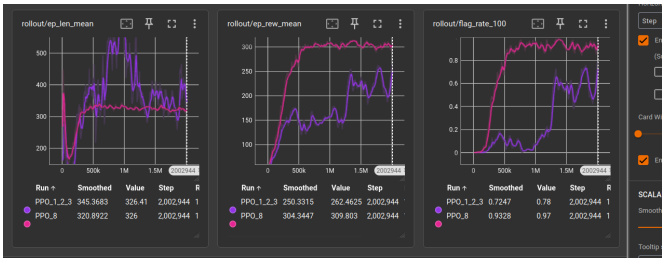

Figure 2: Catastrophic forgetting during 1-1 → 1-2 transfer.

4.4. Random-Start Curriculum

To reduce positional overfitting caused by always starting at the beginning of a level, we combined two complementary mechanisms.

Random-start wrapper. At every episode reset, a `RandomStartWrapper` advances Mario by $n \sim \mathcal{U}[0, N_{\max}]$ steps *before* handing control to the agent. Crucially, Mario is not teleported: the wrapper executes n forward-biased actions sampled from {right, right+A, right+B, right+A+B}, so he physically runs and jumps into the level. If he dies during this warm-up, the episode is reset from position 0 instead.

Curriculum schedule. Rather than fixing $N_{\max} = 100$ from the start (which can be too hard for an untrained policy), a `CurriculumCallback` linearly anneals $N_{\max}$ from 0 to 100 over the course of training, updating the wrapper every 1 000 steps. This two-layer design separates *what* randomisation does (the wrapper) from *when* it is applied (the curriculum), letting the agent first master level entry before facing diverse start positions.

Training on World 1-1 with this scheme produced an agent that generalizes across diverse level sections rather than memorizing a fixed trajectory, evidenced by stable flag rates even when the start position is randomized at evaluation time.

4.5. Multi-Task Training: Worlds 1-1 to 1-4

We trained a single shared PPO policy on Worlds 1-1 through 1-4 simultaneously using 4 parallel environments per level (16 total). The agent converged on World 1-1 while stagnating on the harder levels. The root cause is **reward imbalance**: World 1-1 delivers dense reward signals earliest, so its gradients dominate the policy update and the optimizer exploits the easiest task [6]. Per-task reward normalization or gradient surgery would be required to balance training.

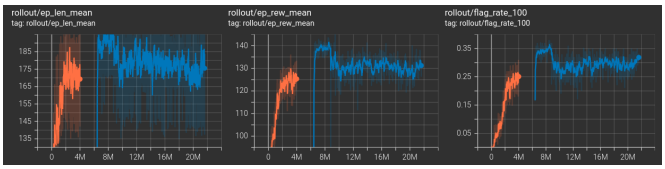

Figure 3: Multi-task training: World 1-1 converges while later levels stagnate.

4.6. Comparison Summary

Table 2 summarizes final evaluation results (30 episodes, deterministic policy).

Table 2: Agent performance on World 1-1 after full training.

Agent	Reward	Flag%	FPS
Symbolic DQN	≈ -35	0%	∼300
Pixel PPO	≈ 285	>80%	∼78
Symbolic PPO	≈ 304	**93%**	**∼500**

5. Discussion

Algorithm > representation. With identical symbolic observations, DQN achieves 0% while PPO reaches 93%. The bottleneck is the algorithm, not the state representation: PPO's clipped objective (2) prevents value divergence, and 8-environment parallelism provides diverse rollouts that GAE can use to propagate the sparse flag reward backwards through long trajectories.

Symbolic efficiency. The 34× observation compression eliminates CNN processing and enables ≈ 6× faster environment throughput. On a CPU alone, the symbolic agent trains to >90% flag rate in under 20 minutes—a practical advantage for rapid prototyping. The trade-off is fragility: symbolic observations depend on a correct NES RAM map and cannot generalize to visually novel environments.

Forgetting and multi-task limits. Catastrophic forgetting is inherent to on-policy fine-tuning without explicit replay [5]. Remedies include Elastic Weight Consolidation (EWC), shared experience replay across tasks, or population-based task scheduling. Multi-task reward imbalance requires explicit per-task normalization or gradient surgery [6] to prevent easy tasks from dominating optimization.

6. Conclusion

We showed that PPO with compact symbolic (RAM-based) observations achieves 93% flag-reach rate on Super Mario Bros 1-1 while training 6× faster than pixel-based PPO on CPU alone. Vanilla DQN fails entirely on this domain, confirming the algorithmic gap. Transfer learning to World 1-2 succeeds at 78% flag rate but causes catastrophic forgetting of the source task. A random-start curriculum mitigates positional overfitting. Naive multi-task training collapses to the easiest level due to reward imbalance.

Future work should apply EWC for forgetting mitigation, per-task reward normalization for multi-task training, and extend the symbolic map to additional game worlds for broader generalization.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.

[2] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[3] A. Raffin et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268):1–8, 2021.

[4] M. McCloskey, N. J. Cohen. Catastrophic Interference in Connectionist Networks. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[5] J. Kirkpatrick et al. Overcoming Catastrophic Forgetting in Neural Networks. *PNAS*, 114(13):3521–3526, 2017.

[6] T. Yu et al. Gradient Surgery for Multi-Task Learning. *NeurIPS*, 2020.